
Can This Benchmark Really Count That Result?

Evidence Envelopes for Measurement Uncertainty in Agent Benchmarks

Shanshan Gao
The University of Sydney
Sydney, Australia

Liyi Zhou
The University of Sydney
Sydney, Australia

Abstract

Interactive-agent benchmarks often report one score over executed tasks. The score can look precise even when the stored artifacts do not fully decide the claim being reported. We study this gap. For each completed (case, agent) record, a predeclared evidence contract asks whether the benchmark’s own task specification and evaluator semantics are sufficient to support a binary result. A record is then reported as counted-SUCCESS, counted-FAIL, or UNRESOLVE. The purpose is not to create a harder task, replace the native evaluator, or build another leaderboard. The purpose is to show what the benchmark can still say when some completed records are not decidable from the available evidence.

Records labeled UNRESOLVE induce an evidence envelope over the fixed completed-record denominator. If S is the number of counted successes, F the number of counted fails, U the number of records labeled UNRESOLVE, and $N = S + F + U$, then the benchmark supports

$$[S/N, (S + U)/N].$$

The width is exactly U/N . The counted-only score $S/(S + F)$ is therefore reported as a diagnostic over the decidable subset, not as a headline estimate of all-record agent capability.

We evaluate this reporting method on four public interactive-agent domains: AGENTDOJO and τ^3 -bench retail as evidence-sensitive domains, and APPWORLD and WEBARENA-VERIFIED as strong-evaluator negative controls. The manifest uses 100 case units per domain crossed with four fixed agents, giving 400 case units, 1600 completed-record slots, and 2000 stored execution episodes because AGENTDOJO case units have benign and injected arms. Evidence contracts are drafted before scoring by a separate cost-aware, high-performing LLM from the task, benchmark specification, evaluator semantics, and schema; human reviewers then audit, edit, and lock the contracts before any agent trace is scored. Predictions about envelope width are registered before scoring, including a falsification clause: large widths on the negative controls would indicate that our evidence requirements are over-conservative rather than that those benchmarks have a newly discovered measurement problem.

The resulting artifact is a measurement report. It contains completed-record denominators, counted successes, counted fails, records labeled UNRESOLVE, coverage, counted-only score, lower and upper evidence bounds, envelope width, pairwise robust-ranking matrices, typed UNRESOLVE reasons, stratified audit checks, and a rerun stability check. We release manifests, evidence contracts, case cards, the UNRESOLVE taxonomy, and a rescorer so that benchmark conclusions can be read together with the evidence envelope induced by records that remain undecidable.

1 Introduction

Suppose a web or tool-use agent is asked to update a customer’s shipping address. The trace shows the agent opening a page, editing a form, and clicking Save. The benchmark emits success. But what exactly was verified? Did the backend record change? Was the intended customer updated? Was the native task asking only for the target address to change, or did it also claim that unrelated state was preserved? If the stored artifacts cannot answer those questions, the trace may be plausible without fully supporting the reported claim.

This paper studies the missing step between execution and reporting. We are not asking whether a benchmark is useful in the broad sense, nor are we proposing a new environment or a universal benchmark standard. We address one narrower question:

When an interactive-agent benchmark cannot decide some completed records from stored evidence, what is the benchmark still entitled to report about performance?

Our answer is deliberately conservative. A completed record should enter a binary success/fail count only when a predeclared claim can be decided from the available evidence. Otherwise the record remains visible as UNRESOLVE, with a typed blocking reason. Records labeled UNRESOLVE are not successes, not fails, and not removed. They determine how wide the benchmark’s measurement envelope is.

From point score to evidence envelope. A common response to undecidable records is to report a score only on the records that can be decided. That quantity is useful, but only conditionally. If decidability depends on the agent’s behavior, then $S/(S + F)$ can become a post-selection statistic over “tasks whose outcome happened to be observable,” not an all-record performance estimate. For example, an agent that leaves decisive evidence only on a few easy successes could score 100% on the counted subset while most of its workload remains unmeasured. We therefore report an *evidence envelope* over the fixed completed-record denominator:

$$\text{Perf}(a, d) \in \left[\frac{S_{a,d}}{N_{a,d}}, \frac{S_{a,d} + U_{a,d}}{N_{a,d}} \right].$$

The lower endpoint treats every record labeled UNRESOLVE as not supporting success; the upper endpoint treats every record labeled UNRESOLVE as potentially successful. The width is $U_{a,d}/N_{a,d}$, the UNRESOLVE share. The envelope is not a confidence interval. It is a partial-identification or reporting-sensitivity interval induced by missing decisive evidence.

Evidence-aware reporting, not a new benchmark claim. The method is layered on top of existing benchmarks. It does not replace tasks, agents, environments, or native verifiers. It asks whether the benchmark’s own success claim is decidable from the stored artifacts. This is important because extra evidence requirements can accidentally turn a benchmark into a stricter task. We avoid that by separating native-aligned claims from stronger measurement claims.

When applying the method to another benchmark, we as adapter authors define the required evidence before scoring, but the definition must be derived from that benchmark’s own task specification and evaluator semantics. The goal is not to invent a harder task, but to ask whether the benchmark’s stated success claim is actually decidable from the stored artifacts. If a required-evidence definition goes beyond the benchmark’s native success claim, we label it explicitly as a stronger measurement claim, report it separately from the native-aligned result, and do not treat resulting disagreements as errors in the original benchmark.

Negative controls and the falsification clause. The same lens clarifies the role of verified benchmarks. Benchmarks such as WEBARENA-VERIFIED and APPWORLD already invest in stronger evaluators. In this study they are not adversaries; they are negative controls. A cross-benchmark reporting method is credible only if it yields narrow envelopes on benchmarks whose artifacts already decide their stated claims. We therefore make the falsification logic explicit: large UNRESOLVE shares on APPWORLD or WEBARENA-VERIFIED would indicate over-conservatism in our evidence contracts, not a new discovery about those benchmarks.

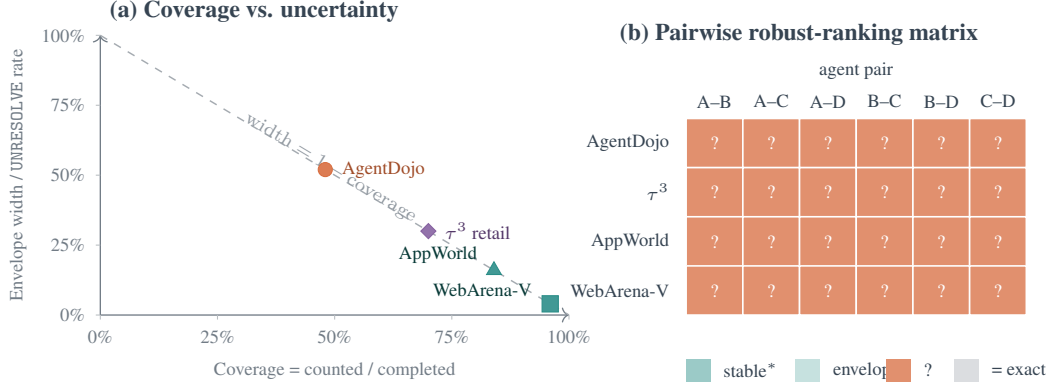


Figure 1: Main reporting figures. (a) The x-axis is coverage $(S + F)/N$; the y-axis is interval width, equivalently the UNRESOLVE rate U/N . Because width equals one minus coverage by definition, the plot is a denominator and evidence-support diagnostic rather than a fitted correlation. (b) Directional pairwise rankings under the evidence envelope. A cell $i > j$ means that agent i 's lower bound exceeds agent j 's upper bound; ? means the ordering is not identified from available evidence. An asterisk marks unadjusted case-resampling stability, not statistical significance. It is added only when the observed envelope is separated and the case-cluster bootstrap lower 2.5th percentile of the dominance margin remains positive.

84 **Domain choice.** We choose four domains for contrast rather than breadth. AGENTDOJO is the
85 clearest evidence-sensitive case because paired benign and injected arms interact with utility and
86 security claims. τ^3 -bench retail is a repaired retail benchmark in the τ -Bench family; the relevant
87 question is what residual measurement uncertainty remains after stronger backend-state and policy
88 checking. APPWORLD and WEBARENA-VERIFIED serve as strong-evaluator negative controls
89 because their native artifacts already track many task claims closely.

90 **Headline reporting objects.** Figure ?? contains the two objects requested by the measurement
91 framing. Panel (a) is a coverage–uncertainty diagnostic: the x-axis is coverage, and the y-axis is
92 interval width, equivalently the UNRESOLVE rate. By construction, width equals one minus coverage,
93 so the plot is not used as a correlation claim. Its purpose is to make the denominator visible. Panel (b)
94 is a pairwise robust-ranking matrix: a cell such as $A > B$ means that Agent A's lower evidence
95 bound exceeds Agent B's upper evidence bound, while ? means that the ordering is not identified
96 from the stored evidence.

97 The paper makes five contributions.

- 98 • We introduce an evidence-aware reporting method for interactive-agent benchmarks: com-
99 pleted records are reported as counted-SUCCESS, counted-FAIL, or UNRESOLVE under a
100 predeclared evidence contract.
- 101 • We show how records labeled UNRESOLVE induce an evidence envelope $[S/N, (S + U)/N]$
102 for all-record performance. The counted-only score $S/(S + F)$ is retained as a diagnostic,
103 not as a headline all-record estimate.
- 104 • We provide a seven-category UNRESOLVE taxonomy (Section ??), explicitly framed as the
105 minimum stable set covering this study rather than as a universal vocabulary.
- 106 • We present a fixed four-domain study over AGENTDOJO, APPWORLD, WEBARENA-
107 VERIFIED, and τ^3 -bench retail, scaled to 100 case units per domain, with pre-registered
108 predictions, strong-evaluator negative controls, a rerun stability check, and a blinded two-
109 auditor study.
- 110 • We provide an E&D-oriented release package: predeclared manifests, evidence contracts,
111 case cards, UNRESOLVE-reason records, robust-ranking matrices, and a rescorer that makes
112 denominators and evidence envelopes auditable.

113 The intended lesson is practical. We do not claim that every interactive benchmark has a large
114 evidence problem. We claim that a benchmark result is incomplete unless it states which records

115 were allowed to count, which records remained UNRESOLVE, and how wide the resulting evidence
116 envelope is.

117 2 Evidence-Supported Counting

118 2.1 Units: case units, records, episodes, evidence

119 A *case* specifies a task, the initial environment, the benchmark claim being tested, and the evidence
120 required to decide that claim. The reporting unit at the manifest level is the *case unit*: in most
121 domains, one task instance is one case unit; in AGENTDOJO, a benign arm and an injected arm form
122 one paired case unit even though the arms are executed separately.

123 Decisions live at the level of *records*. A record is a (case unit, agent) pair. With 400 main-study case
124 units and four agents, the main study contains 1600 record slots. Each completed record is what the
125 counting rule decides on. The distinction matters because the same case unit can be countable for one
126 agent, whose trace exposes decisive evidence, and UNRESOLVE for another, whose trace does not.

127 We separate two ways a record can be labeled UNRESOLVE. *Trace-level UNRESOLVE* means that
128 the benchmark could in principle decide the claim from a sufficiently rich trace, but this particular
129 trace does not expose the needed evidence. *Instrument-level UNRESOLVE* means that the benchmark
130 artifacts do not provide a way to support the stated claim from any trace. The envelope formulas
131 below are partial-identification bounds for trace-level UNRESOLVE and reporting-sensitivity bounds
132 for instrument-level UNRESOLVE.

133 An *episode* is one concrete execution of one agent on one runnable arm of a case. Episodes produce
134 *traces*: ordered records of actions, observations, tool calls, browser events, messages, files, and
135 post-run artifacts. *Evidence* is the trace subset or post-run measurement that can support the active
136 claim. A trace can be plausible without being decisive.

137 2.2 Native-aligned evidence contracts

138 Each case unit carries a predeclared evidence contract. The contract states: (i) the native benchmark
139 claim; (ii) the official source of that claim, such as the task text, evaluator code, policy, or schema;
140 (iii) the artifacts required to decide success or fail; (iv) the rule for assigning UNRESOLVE; and (v)
141 whether the claim is native-aligned or a stronger measurement claim.

142 The contract must be minimal with respect to the benchmark’s own semantics. For example, if a task
143 says only “update the address” and the native evaluator only checks the target record’s address, then
144 the native-aligned contract asks for target-record identity and post-state evidence. A no-collateral-
145 change requirement is included only if the benchmark’s own task, policy, evaluator, or reported claim
146 requires it. Otherwise it is labeled as a stronger measurement claim and reported separately.

147 When sources differ, the contract follows a fixed source hierarchy: official evaluator semantics first;
148 official task text and policy second; and schema constraints needed to interpret evaluator-visible state
149 third. We do not add requirements from annotator intuition alone. Any requirement not grounded in
150 one of these sources is labeled as a stronger measurement claim and excluded from the native-aligned
151 main envelope.

152 We use a separate contract-drafting LLM before scoring. In the main study, the contract drafter
153 was FILL, queried with temperature FILL using prompt version FILL. The drafter receives the task,
154 benchmark specification, evaluator semantics, database or API schema, and trace format. It does
155 not receive agent identities, agent traces, native scores, or any outcome labels. The drafter is used
156 only to propose evidence requirements. Human adapter authors then audit, edit, and lock each
157 contract before scoring starts. They source-check every proposed requirement against the official
158 task text, evaluator, policy, or schema. Requirements without such support are removed or labeled
159 as stronger measurement claims. The locked manifest records the drafter model identifier, prompt
160 version, temperature, human reviewer, contract version, and claim-scope label.

161 2.3 Counting rule

162 A completed record $u = (c, a)$, for case unit c and agent a , is *counted* when three conditions hold: (i)
163 the relevant execution ran; (ii) the claim to be tested was predeclared; and (iii) the available evidence

is sufficient to assign SUCCESS or FAIL. A record is UNRESOLVE if it is relevant to the benchmark but the claim cannot be decided because a required detector, state query, receipt, identity mapping, state-preservation check, or evidence path is missing or ambiguous.

The rule assigns every completed record to exactly one of three observable categories: counted-SUCCESS, counted-FAIL, or UNRESOLVE. Execution failures are not merged into the UNRESOLVE label. Only infrastructure or pre-run failures that prevent normal benchmark execution are excluded from the completed-record denominator. Agent-caused invalid actions, timeouts, tool misuse, or aborts after normal benchmark execution has begun are counted as FAIL when the native benchmark would treat them as failure, or are reported in an all-attempt denominator. They are not silently removed.

2.4 From counted records to evidence envelopes

For an agent a and domain d , let $R(a, d)$ be the fixed set of completed records eligible for evidence analysis and let $N_{a,d} = |R(a, d)|$. Under the counting rule, let $S_{a,d}$, $F_{a,d}$, and $U_{a,d}$ denote the counts of counted-SUCCESS, counted-FAIL, and UNRESOLVE records, with

$$N_{a,d} = S_{a,d} + F_{a,d} + U_{a,d}.$$

The counted-only success rate is

$$\text{CountedScore}_r(a, d) = \frac{S_{a,d}}{S_{a,d} + F_{a,d}},$$

when $S_{a,d} + F_{a,d} > 0$. This quantity is conditional on decidability. It is not interpreted as an unbiased estimate of all-record agent performance when $U_{a,d} > 0$, because decidability can be agent-dependent.

For the all-record binary performance claim, records labeled UNRESOLVE imply an evidence envelope:

$$\text{LB}_r(a, d) = \frac{S_{a,d}}{N_{a,d}}, \quad \text{UB}_r(a, d) = \frac{S_{a,d} + U_{a,d}}{N_{a,d}}.$$

Thus the benchmark supports

$$\text{Perf}_r(a, d) \in [\text{LB}_r(a, d), \text{UB}_r(a, d)].$$

The envelope width is

$$\text{Width}_r(a, d) = \text{UB}_r(a, d) - \text{LB}_r(a, d) = \frac{U_{a,d}}{N_{a,d}},$$

and coverage is

$$\text{Coverage}_r(a, d) = \frac{S_{a,d} + F_{a,d}}{N_{a,d}}.$$

The lower and upper endpoints are the pessimistic and optimistic resolutions of records that remain undecidable from the stored evidence. They are not confidence limits.

2.5 What the case-resampling intervals mean

The main manifest is selected by deterministic hash order, not by a random sampling device. For the fixed manifest, S , F , U , coverage, and the evidence envelope are finite-manifest descriptions. They do not need confidence intervals to be meaningful.

We nevertheless report case-cluster bootstrap intervals as a secondary view. Their working estimand is the corresponding mean quantity over an eligible benchmark case-unit population. This interpretation requires the selected case units to be treated as exchangeable with draws from that population after the predeclared eligibility filter. Readers who do not accept this case-population assumption should read the point estimates as the primary results and the intervals as descriptive resampling diagnostics.

The bootstrap is clustered by case unit. Each replicate resamples case units with replacement within domain and keeps all agent records attached to the resampled case unit. This is necessary because the four agent records attached to one case unit can be strongly correlated. We do not use record-level independence or a 400-record binomial standard error as a formal precision claim.

The intervals also condition on the stored traces and fixed agent versions. They do not quantify total uncertainty over repeated agent executions, temperature randomness, API drift, tool-call nondeterminism, or environment volatility. The rerun subset in Section ?? is a stability check for the evidence labels; it is not an agent-performance confidence interval.

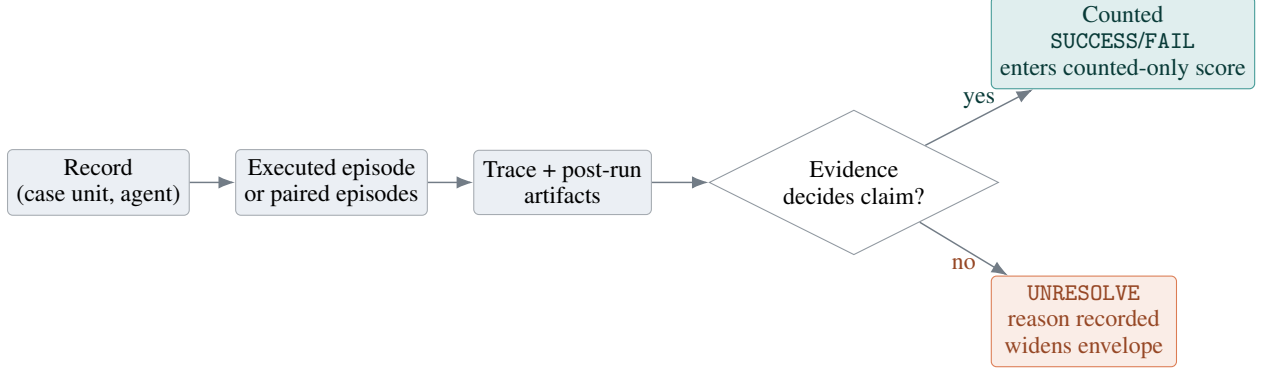


Figure 2: Evidence-supported counting as a process. Counted records support binary labels; records labeled UNRESOLVE remain in the fixed completed denominator and widen the evidence envelope.

2.6 Robust pairwise rankings

The envelope changes how pairwise agent comparisons are reported. For two agents i and j in domain d , the supported difference interval is

$$\Delta(i, j; d) = [LB_{i,d} - UB_{j,d}, UB_{i,d} - LB_{j,d}].$$

The observed-envelope decision rule is directional:

$$LB_{i,d} > UB_{j,d} \Rightarrow i \succ j, \quad LB_{j,d} > UB_{i,d} \Rightarrow j \succ i.$$

If neither inequality holds, the pair is *not identified*: the available evidence supports resolutions under which either agent could be at least as good as the other. We use “=” only for exact equality of the two envelopes up to numerical tolerance; ordinary overlap is reported as “?” rather than as a tie.

For an observed ordering $i \succ j$, the dominance margin is

$$M_{i>j} = LB_{i,d} - UB_{j,d}.$$

We add an asterisk only when the 2.5th percentile of the case-cluster bootstrap distribution of $M_{i>j}$ remains positive. The symmetric criterion applies when $j \succ i$.

This asterisk is a descriptive, unadjusted case-resampling stability tag. It is not a formal significance test. The tag is applied after the observed envelope separation has already selected the pair, and the main study contains $4 \text{ domains} \times 6 \text{ pairs} = 24$ pairwise comparisons. We therefore do not interpret the tag as controlling family-wise error or false discovery rate.

2.7 Comparison views on the same traces

We compare the evidence envelope with diagnostic point-score views applied to the same stored traces. These views are useful for explaining how published scores move, but the envelope is the headline measurement object.

We also separate two sources of disagreement. A *verdict shift* occurs when two views count the same record but assign different success/fail outcomes. A *countability shift* occurs when one view lets a record affect a point score but the evidence rule reports UNRESOLVE. The latter is not a nuisance to be hidden; it is what widens the evidence envelope.

2.8 The UNRESOLVE taxonomy: seven blocking reasons

A UNRESOLVE label is informative only if it explains what is missing. We collapse blocking reasons across the four main domains into a seven-category taxonomy (Table ??). We do not claim that these categories are exhaustive across all future interactive-agent benchmarks. We claim that they are the minimum stable set under which the auditors in Section ?? can classify every record labeled UNRESOLVE in this study. The categories overlap by design within the broader bucket “evidence missing.” We resolve overlap with an upstream-priority rule: when a record matches more than one category, it is tagged with the upstream reason that, if resolved first, would unblock the others.

Table 1: Reporting views used in the paper. The evidence envelope is the headline object. Point scores are diagnostics and are never treated as unbiased all-record performance estimates when records labeled UNRESOLVE exist.

View	Denominator	Undecidable-record handling	Purpose
Evidence envelope	Completed records	Bounds $[S/N, (S + U)/N]$	Headline measurement uncertainty induced by records that cannot be decided from stored evidence
Counted-only score	Counted records only	Excluded from the point score, but reported as UNRESOLVE	Diagnostic success rate conditional on decidability; not all-record performance
Native benchmark	Benchmark default	Whatever the released benchmark reports	Compare against what a user would normally publish
Optimistic endpoint	Completed records	UNRESOLVE $\rightarrow$ success-supporting	Upper endpoint of the evidence envelope
Pessimistic endpoint	Completed records	UNRESOLVE $\rightarrow$ not success-supporting	Lower endpoint of the evidence envelope
Judge-only (appendix)	Judge-readable records	Judge may assign SUCCESS/FAIL/INCONCLUSIVE	Diagnostic comparison to semantic trace judgment

Evidence-sensitive record (τ^3 -bench retail)	Negative-control record (AppWorld)
Task Update the intended customer's mailing address to "123 Maple St." Native-aligned claim The intended customer record has its address updated. Available evidence Tool call <code>update_address(...)</code>; assistant message "address updated." Missing piece No post-update backend query; the trace also lacks a unique customer id, so target identity cannot be verified. Counting decision UNRESOLVE (R3 upstream over R1). Scope guardrail A no-collateral-change check would be reported only if required by the native task or as a separate stronger measurement claim. Effect on report The record stays in N, contributes one unit to U, and widens $[S/N, (S + U)/N]$.	Task Add a \$42.50 grocery expense from yesterday to the expenses tracker. Native-aligned claim New row in expenses with amount 42.50, category "groceries," and correct date, under the native unit-test semantics. Available evidence Post-run database snapshot and unit-test fixture that directly checks the relevant fields. Missing piece None for this trace under the native-aligned claim. Counting decision Counted, verdict SUCCESS. Effect on report The record contributes one unit to S, appears in the counted-only score, and does not widen the envelope.

Figure 3: Two records walked through the method. A record labeled UNRESOLVE is not a fail, not a success, and not removed; it widens the measurement envelope. The example also shows the native-alignment guardrail: stronger checks are not silently folded into the native-aligned result.

2.9 Worked example: one UNRESOLVE record widens the envelope

Figure ?? walks through two representative records side by side. The key change from a score-only view is that the record labeled UNRESOLVE on the left is not deleted from the measurement. It remains in N , increases U , and widens the domain's envelope by $1/N$.

3 Study Design

3.1 Why these domains

The study uses four main domains chosen to make the evidence claim falsifiable. A method that only finds large gaps is not informative. We therefore mix evidence-sensitive and strong-evaluator domains.

3.2 Agents, manifests, and fixed workload

The main study uses four fixed agents, Agent A, Agent B, Agent C, and Agent D, selected before scoring as non-redundant measurement probes spanning source openness, scale, and tool-use style. Four probes cannot exhaust a factorial design; they are not meant to be a leaderboard. They are used

Table 2: UNRESOLVE-reason taxonomy. Every record labeled UNRESOLVE is tagged with exactly one category; the upstream-priority rule disambiguates overlap.

UNRESOLVE reason	What is missing	Representative example
R1: Missing state query	The native-aligned claim depends on a backend or environment state value that was never queried or stored.	APPWORLD task expects a row added to a CRM table; trace contains no read of that table after the action.
R2: Unobservable side effect	The action is performed, but its consequence, such as delivery, send, payment, or receipt, is not exposed in any artifact.	AGENTDOJO agent appears to send a confirmation, but the messaging tool returns no receipt or message-id.
R3: Ambiguous identity mapping	Multiple records or recipients match the claim and the trace does not pin down which one was acted on.	τ^3 -bench retail task asks to update a customer’s address; two customers with matching name exist, and no unique id is logged.
R4: Required state-preservation evidence absent	The benchmark’s own task, policy, evaluator, or reported claim requires non-modification of unrelated state, but no diff or snapshot evidence is available.	A native task requires editing one calendar event and preserving all other events, but no post-run diff over other events is captured.
R5: Paired-arm asymmetry	A paired case requires both arms to be decidable; one arm is decidable and the other is not.	AGENTDOJO benign arm has clean state-diff evidence; injected arm’s security claim is undecidable.
R6: Evaluator output ambiguity	The native evaluator emits a label, but its inputs do not uniquely determine that label under the predeclared native-aligned claim.	WEBARENA-VERIFIED verifier accepts a string match, but the stored verifier inputs do not show whether the required structured target was reached.
R7: Claim-scope mismatch	The trace shows the agent solving a related task that does not match the predeclared claim, with no evidence on the claim actually being reported.	τ^3 -bench retail agent gives a semantically helpful refund response, but the policy-relevant backend record required by the benchmark claim is absent.

Table 3: Main domains. The number of records is (case units) $\times$ four agents. AGENTDOJO has two episodes per record because each case unit has a benign and injected arm.

Domain	Role in paper	Case units	Records	Episodes	Native signal	Evidence sources
AGENTDOJO	Evidence-sensitive positive example	100 paired	400	800	utility/security label	workspace state across both arms, tool calls, files, messages
APPWORLD	Strong negative control	100	400	400	database-state unit tests	database state, API logs, unit-test artifacts, native field checks
WEBARENA-VERIFIED	Strong negative control	100	400	400	verified deterministic evaluator	browser artifacts, network trace, structured final output, verifier inputs
τ^3 -bench retail	Residual repaired-benchmark domain	100	400	400	native success signal	database state, tool records, policy-relevant backend evidence
Main study total	–	400	1600	2000	–	–
ANDROIDWORLD	Appendix negative control	—	—	—	task-specific success check	device/system state and evaluator artifacts
WORKARENA	Appendix negative control	—	—	—	benchmark validators	browser state and enterprise-workflow artifacts
OSWORLD-VERIFIED	Appendix stress test	—	—	—	verified evaluator	separated execution-failure, evaluator-failure, and evidence diagnostics

247 to test whether evidence uncertainty is structured by domain rather than by a single agent family.
248 Concrete model identifiers and version pins are released with the manifests.

249 The manifest selects 100 eligible case units per domain using a deterministic, predeclared hash
250 order after excluding smoke-test cases. With four agents this gives 1600 record slots. Because
251 each AGENTDOJO record runs a benign and an injected arm, the main study stores 2000 execution
252 episodes. If a domain has fewer than 100 eligible verified cases under the official split, the manifest
253 uses all eligible cases and records the exception before main scoring.

254 **Denominator rule.** Only infrastructure or pre-run failures are excluded from the completed-
255 record denominator. Examples include a sandbox that fails to boot before the task is presented, a
256 missing benchmark asset, or an external service outage that prevents normal benchmark execution.
257 Agent-caused invalid actions, tool misuse, timeouts after task start, malformed final answers, or
258 benchmark-facing aborts are not excluded. They are counted as FAIL when the native benchmark
259 would fail the run, and they are also reported in the all-attempt denominator audit in Table ??.

260 separation prevents evidence filtering from raising scores by removing difficult or agent-induced
261 failures.

262 **Sample size and precision.** The primary unit for uncertainty is the case unit, not the record. Each
263 domain has 100 case units crossed with four agents, and the four records attached to a case unit can
264 be correlated because the agents face the same task. We therefore avoid formal precision claims based
265 on 400 independent records per domain. A record-level binomial calculation would be, at best, an
266 independence lower bound and is not used for inference. All resampling intervals are case-clustered
267 and condition on the stored traces.

268 3.3 Evidence-contract drafting and locking

269 For every case unit, the contract-drafting LLM receives:

- 270 • the task text and any official policy attached to the task;
- 271 • the benchmark’s evaluator code or evaluator description;
- 272 • database, API, browser, file, or tool schemas relevant to the task;
- 273 • the trace schema and available post-run artifact types; and
- 274 • a template requiring the drafter to distinguish native-aligned evidence from stronger mea-
275 surement evidence.

276 The drafter proposes a structured contract:

```
277 claim_text: ...  
278 native_sources: [task_text, evaluator, policy, schema]  
279 required_artifacts: ...  
280 success_rule: ...  
281 fail_rule: ...  
282 unresolve_rule: ...  
283 claim_scope: native_aligned | stronger_measurement  
284 minimality_rationale: ...  
285 source_support:  
286   evaluator: ...  
287   task_or_policy: ...  
288   schema: ...  
289
```

291 Human adapter authors then review the contract. They check whether each required artifact is derived
292 from the native benchmark semantics, whether any stronger requirement is marked separately, and
293 whether the UNRESOLVE rule is inspectable. They apply the fixed source hierarchy in Section ??:
294 evaluator semantics, then task text and policy, then schema constraints needed to interpret evaluator-
295 visible state. The contract is locked before any agent trace is scored. After locking, neither the LLM
296 nor the human reviewer may alter the contract based on agent outcomes. Any later clarification is
297 versioned and reported as a sensitivity analysis, not folded into the main result.

298 3.4 Pre-registered predictions and the falsification clause

299 To make the main claim falsifiable, we register predictions before scoring any record. Predictions are
300 stored verbatim in the manifest and are not edited after the first scoring run. The prediction target is
301 envelope width, which equals the UNRESOLVE share.

302 Each prediction is evaluated in a three-state way under the case-population interpretation described
303 in Section ?. A prediction is *supported* only when the relevant 95% case-cluster bootstrap interval
304 is entirely on the predicted side of the threshold or contrast. It is *falsified* only when the interval
305 is entirely on the opposite side. It is *inconclusive* otherwise. Point estimates are still reported for
306 transparency, but they do not alone determine support or falsification.

- 307 • **P1 (negative-control bound + falsification clause).** Per-record envelope width on APP-
308 WORLD and on WEBARENA-VERIFIED is each below 15%. Supported if both upper 95%
309 case-resampling bounds are below 15%; falsified if either lower 95% case-resampling bound
310 is above 15%; otherwise inconclusive. A falsification indicates that the evidence contracts
311 are likely over-conservative for strong-evaluator benchmarks.

- 312 • **P2 (evidence-sensitive bound).** Per-record envelope width on AGENTDOJO exceeds the
313 maximum of the two negative-control widths by at least 20 percentage points. Supported
314 if the lower 95% case-resampling bound of the contrast exceeds 20 percentage points;
315 falsified if the upper 95% case-resampling bound is below 20 percentage points; otherwise
316 inconclusive.
- 317 • **P3 (residual band).** Per-record envelope width on τ^3 -bench retail is strictly above each
318 negative-control width and strictly below AGENTDOJO’s width. Supported if both contrast
319 intervals are entirely in the predicted direction; falsified if either contrast interval is entirely
320 in the opposite direction; otherwise inconclusive.
- 321 • **P4 (ranking uncertainty concentration).** At least 75% of pairwise agent comparisons not
322 identified by the evidence envelope occur on AGENTDOJO or τ^3 -bench retail. Supported
323 if the lower 95% case-resampling bound for that fraction is at least 75%; falsified if the
324 upper 95% case-resampling bound is below 75%; otherwise inconclusive. Exact zero-width
325 equalities are reported separately and are not counted as evidence-induced non-identification.

326 P1 is the calibration check. The method earns its claim only if it produces narrow envelopes on
327 strong-evaluator negative controls. P2–P4 test whether wide envelopes and non-identified rankings
328 concentrate in the domains where the predeclared claims are expected to outrun available evidence.

329 3.5 Domain evidence sources

330 Each case unit carries a locked evidence contract. In APPWORLD, the contract centers on database
331 state, API call history, and unit-test artifacts. In WEBARENA-VERIFIED, it centers on verified
332 evaluator inputs, browser artifacts, and network-trace evidence. In τ^3 -bench retail, it centers on
333 tool logs, backend state, policy-relevant records, and identity resolution. In AGENTDOJO, it centers
334 on workspace state across benign and injected arms, messages, files, tool calls, and the paired
335 security/utility condition.

336 The scorer never invents a post hoc claim. It asks whether the locked, native-aligned claim can
337 be decided from stored evidence. If not, the record is UNRESOLVE even when the trajectory looks
338 plausible.

339 3.6 Audit and rerun checks

340 We run two fixed checks on the main domains.

341 **Rerun subset.** We rerun Agent A on ten case units per domain, giving 40 rerun case units and
342 50 rerun episodes because AGENTDOJO reruns paired arms. We report agreement with the original
343 counting decision and agreement in envelope category (counted success, counted fail, UNRESOLVE
344 reason), together with exact (Clopper–Pearson) 95% binomial intervals. We also report the absolute
345 change in lower endpoint, upper endpoint, and width on the rerun subset. This check only probes
346 stability of evidence labels under a small fixed rerun. It does not support confidence intervals for
347 agent performance under repeated executions.

348 **Blinded human audit.** We audit 100 main-study records, 25 per domain. Within each domain,
349 the sample is stratified into counted records, records labeled UNRESOLVE, and records where native
350 reporting and evidence reporting disagree whenever all three buckets exist. This is a stress sample for
351 inspectability, not a simple random sample. The raw agreement rates and Cohen’s κ values therefore
352 do not estimate population-level agreement unless design weights are applied. We report the stratum
353 counts for every domain.

354 For each audit item, the auditor sees the task, the trace, available evidence, and the locked evidence
355 contract. The auditor does not see the agent identity, native benchmark label, counting decision,
356 UNRESOLVE-reason tag, or alternate view verdicts. Two auditors independently classify every item.
357 We compute:

- 358 • auditor-vs.-scorer agreement on {counted-SUCCESS, counted-FAIL, UNRESOLVE};
- 359 • inter-rater Cohen’s κ on {counted, UNRESOLVE};

Table 4: Main measurement report. Counts are over completed records after excluding only infrastructure/pre-run failures. Total $N = S + F + U$. Coverage is $(S + F)/N$. Counted-only score is $S/(S + F)$, a conditional diagnostic over decidable records. Lower is S/N , Upper is $(S + U)/N$, and Width is U/N . The headline performance object is the interval [Lower, Upper], not Counted-only score alone.

Domain	Total	SUCCESS	FAIL	UNRESOLVE	Coverage	Counted-only score	Lower	Upper	Width
AGENTDOJO	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
APPWORLD	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	400	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL
All domains	1600	FILL	FILL	FILL	FILL	FILL	FILL	FILL	FILL

Table 5: Denominator audit. Only infrastructure or pre-run failures are excluded from the completed-record denominator. Agent-caused aborts, invalid actions, and timeouts after normal benchmark execution starts are counted as FAIL when the native benchmark would fail them, and they remain visible in the all-attempt audit.

Domain	Attempted	Infra/pre-run excluded	Completed	Agent-caused FAIL	Notes
AGENTDOJO	FILL	FILL	400	FILL	Paired benign/injected arms remain linked at the case-unit level.
APPWORLD	FILL	FILL	400	FILL	Native database-state failures remain in the completed denominator.
WEBARENA-V	FILL	FILL	400	FILL	Browser-task timeouts after start remain in the completed denominator.
τ^3 -bench retail	FILL	FILL	400	FILL	Tool misuse and policy-path failures remain in the completed denominator.
All domains	FILL	FILL	1600	FILL	Exclusions are not allowed to depend on whether the agent likely succeeded.

360 • inter-rater Cohen’s κ on $\{R1, \dots, R7\}$, computed only over items both auditors mark
361 UNRESOLVE.

362 The κ values are descriptive. They can be unstable with 25 audit items per domain, and the taxonomy
363 κ may be computed on a smaller subset because it requires both auditors to choose UNRESOLVE. The
364 release therefore includes item-level audit labels and stratum counts. If a population-level agreement
365 estimate is needed, the same labels can be combined with design weights from the main manifest.

366 **Adoption note.** Adopting the method on top of an existing benchmark requires writing one locked
367 claim and evidence contract per case unit, then running the rescorer over stored traces. The rescorer
368 returns S, F, U , coverage, counted-only score, envelope endpoints, interval width, and UNRESOLVE-
369 reason records. Per-domain human-time breakdowns appear in Appendix ??.

370 4 Main Results

371 Across the fixed four-domain study, the evidence rule separates what the benchmark executed from
372 what the benchmark can support as a binary claim. The core result is not a new point score. It is the
373 structure of the evidence envelope: wide where claims outrun evidence, narrow where evaluators
374 already track the claim closely.

375 4.1 Envelope width is concentrated, not universal

376 The informative observation is not merely how many records are labeled UNRESOLVE, but where
377 interval width appears. The registered expectation is that AGENTDOJO and residual τ^3 -bench retail
378 records have wider envelopes, while APPWORLD and WEBARENA-VERIFIED have narrow envelopes.
379 This pattern supports calibration only when the negative controls remain narrow: the evidence rule

Table 6: Descriptive case-resampling intervals for envelope quantities. The evidence envelope itself is induced by records that are undecidable from stored evidence. The 95% intervals are case-cluster bootstrap intervals for the eligible-case population interpretation, conditional on stored traces and fixed agent versions. They do not include repeated-execution randomness, API drift, or environment volatility.

Domain	Lower S/N (95% case-resampling interval)	Upper $(S+U)/N$ (95% case-resampling interval)	Width U/N (95% case-resampling interval)	Counted-only score $S/(S+F)$ (95% case-resampling interval)	Native score inside envelope
AGENTDOJO	FILL	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL	FILL
All domains	FILL	FILL	FILL	FILL	FILL

Table 7: Pre-registered prediction outcomes. The prediction target is envelope width, equivalently the UNRESOLVE rate. Outcomes use the three-state case-resampling interval rule in Section ??: supported, falsified, or inconclusive.

Prediction	Registered criterion	Observed quantity	Outcome
P1	APPWORLD and WEBARENA-VERIFIED widths each below 15% under the case-resampling interval rule	FILL	FILL
P2	AGENTDOJO width exceeds max negative-control width by at least 20 pp under the case-resampling interval rule	FILL	FILL
P3	τ^3 -bench retail width lies above both negative controls and below AGENTDOJO under the case-resampling interval rule	FILL	FILL
P4	At least 75% of non-identified pairs occur on AGENTDOJO or τ^3 -bench retail under the case-resampling interval rule	FILL	FILL

380 should be conservative where evidence is genuinely insufficient, but it should nearly agree with strong
381 evaluators whose artifacts already decide the native-aligned claim.

382 4.2 Counted-only score is diagnostic, not a leaderboard score

383 The counted-only score $S/(S+F)$ remains useful because it says how agents perform on records
384 for which the benchmark can decide the claim. It is not the headline estimate of all-record ability.
385 Table ?? therefore always shows counted-only score next to coverage, lower bound, upper bound, and
386 width. A high counted-only score with low coverage is interpreted as “high success on the decidable
387 subset, low measurement support,” not as a strong all-record agent result.

388 4.3 Robust rankings replace single-order conclusions

389 Pairwise rankings are reported directionally. A cell $i > j$ means that the observed evidence envelopes
390 are separated in favor of agent i ; $i > j^*$ means the same dominance margin also remains positive
391 under the unadjusted 95% case-cluster bootstrap band. A cell ? means the envelopes overlap and the
392 ordering is not identified from available evidence. We reserve “=” for exact equality up to numerical
393 tolerance; overlap is uncertainty, not a tie.

Table 8: Directional pairwise robust-ranking matrix. Four agents induce six pairwise comparisons per domain. A cell $A > B$ is reported only when $LB_{A,d} > UB_{B,d}$; ? denotes envelope overlap. An asterisk marks unadjusted case-resampling stability, not statistical significance. It is added only when the observed envelope is separated and the lower 2.5th percentile of the case-cluster bootstrap dominance margin remains positive.

Domain	A-B	A-C	A-D	B-C	B-D	C-D	Non-identified pairs
AGENTDOJO	?	?	?	?	?	?	FILL
APPWORLD	?	?	?	?	?	?	FILL
WEBARENA-VERIFIED	?	?	?	?	?	?	FILL
τ^3 -bench retail	?	?	?	?	?	?	FILL
All domains	Unsupported native strict orderings: FILL; native conflicts: FILL						FILL

Table 9: Pairwise dominance margins for envelope-separated comparisons. For a reported ordering $i > j$, the margin is $M_{i>j} = LB_{i,d} - UB_{j,d}$. An asterisk marks unadjusted case-resampling stability, not statistical significance. Because the tag is post-selection and unadjusted over 24 comparisons, it should be read as descriptive stability rather than a formal significance test.

Domain	Pair	Envelope decision	Margin and 95% case-resampling interval	Interpretation
FILL FROM SCORED MANIFEST	—	—	—	Include every envelope-separated pair and its case-cluster bootstrap dominance margin.

This distinction also clarifies how to compare native benchmark rankings with evidence-envelope rankings. A native point score can impose a strict order where the evidence envelope says ?. We call that order *unsupported* by the evidence analysis, not a conflict. A *conflict* occurs only when the native strict order is opposite to a robust directional order identified by the envelope. This avoids overstating disagreement: much of the practical lesson is that the benchmark may be unable to support a strict ranking at all, rather than that it supports the opposite ranking.

4.4 Counting decisions are inspectable

The evidence rule is useful only if its decisions are inspectable. The rerun subset and blinded human audit test that property directly.

Disagreements are useful because they reveal where the evidence contract or UNRESOLVE taxonomy still needs clarification. The audit disagreement set is part of the release.

5 Discussion

This paper offers an evidence-aware reporting method rather than a full theory of benchmark validity. The key positioning is that the method does not claim to remove bias and produce a better point score. It exposes the measurement uncertainty that undecidable records create. When $U > 0$, the benchmark supports an envelope, not a single well-identified all-record performance value.

Why this is stronger than a counted-only score. Dropping records labeled UNRESOLVE from the denominator can create a selection problem: the decidable subset may depend on the agent. The counted-only score therefore has a restricted interpretation. The envelope keeps the fixed completed denominator visible and makes the uncertainty caused by undecidable records quantitative. The width U/N is a benchmark-level property of the measurement claim under the locked evidence contract.

Why this does not redefine the benchmark task. The native-alignment guardrail is central. Required evidence must be derived from the benchmark’s own task specification, evaluator semantics, policy, and schema. If an evidence requirement goes beyond the benchmark’s native claim, we

Table 10: Dominant UNRESOLVE reasons by domain. The full UNRESOLVE-reason histogram is released with the case-level manifest; this table reports the largest source of undecidable evidence in each main domain.

Domain	Top UNRESOLVE reason
AGENTDOJO	FILL
APPWORLD	FILL
WEBARENA-VERIFIED	FILL
τ^3 -bench retail	FILL

Table 11: Stratified audit and rerun checks. Audit strata report how many items come from counted records, records labeled UNRESOLVE, and native/evidence disagreement records. Agreement and κ values are descriptive for this stratified stress sample, not unweighted estimates for the full record population. Taxonomy κ uses {R1–R7} and is computed only on items both auditors marked UNRESOLVE. Rerun agreement is a small stability check for evidence labels, not an agent-performance uncertainty estimate.

Domain	Audit items	Audit strata	Audit agreement	Countability κ	Taxonomy κ	Rerun agreement / dominant disagreement
AGENTDOJO	25	FILL	FILL	FILL	FILL	FILL; FILL
APPWORLD	25	FILL	FILL	FILL	FILL	FILL; FILL
WEBARENA-VERIFIED	25	FILL	FILL	FILL	FILL	FILL; FILL
τ^3 -bench retail	25	FILL	FILL	FILL	FILL	FILL; FILL
All domains	100	FILL	FILL	FILL	FILL	FILL; FILL

label it as a stronger measurement claim and report it separately. For example, “address updated correctly” and “address updated correctly with no unrelated field changed” are different claims unless the benchmark itself commits to the second. Disagreement under a stronger measurement claim is not treated as an error in the original benchmark.

Verified benchmarks as calibration checks. APPWORLD and WEBARENA-VERIFIED are not counterexamples; they are calibration tests. Large widths on these domains would falsify the calibration of our evidence contracts. Narrow widths would show that the reporting layer can recognize when native evaluators already make the claim decidable. This behavior is essential for a method meant to be adopted across benchmarks.

Two kinds of uncertainty. The evidence envelope is not a confidence interval. It is induced by stored evidence that cannot decide the claim. Case-cluster bootstrap intervals around the lower endpoint, upper endpoint, width, and coverage quantify a different question: how much the measured quantities would vary under an exchangeable eligible-case-population interpretation, conditional on the stored traces. They do not capture repeated-execution stochasticity. Reporting both avoids conflating unresolved evidence with finite-case resampling.

Practical implication. A benchmark result should report at least nine objects together: total completed records, counted successes, counted fails, records labeled UNRESOLVE, coverage, counted-only score, lower bound, upper bound, and width. Pairwise comparisons should be reported as robust only when the directional dominance margin is positive. Asterisks should be described as unadjusted case-resampling stability tags, not as statistical significance, unless simultaneous intervals or multiple-testing corrections are used.

The paper has limits. First, the result is only as good as the locked claim and evidence contract. A vague claim cannot be rescued by a formal envelope. Second, the seven-category taxonomy is the minimum stable set for this study, not a universal vocabulary. Third, the four-agent design is a measurement probe, not a definitive leaderboard. Fourth, instrument-level UNRESOLVE makes the envelope a sensitivity report rather than a recovered latent ground-truth interval. Fifth, the main bootstrap intervals condition on stored traces and do not capture repeated agent-execution uncertainty.

445 These limits are precisely why the release includes manifests, evidence contracts, taxonomy versions,
446 denominator audits, audit labels, and the rescorer.

447 **Adoption note.** The method is additive to existing benchmark releases. A benchmark that already
448 publishes a verifier can adopt the reporting layer by writing one explicit claim per case and exposing
449 the artifacts required to decide it. The release package accepts existing benchmark trace formats
450 and returns counted/UNRESOLVE decisions, taxonomy tags, coverage, counted-only score, envelope
451 endpoints, interval width, denominator audits, and robust-ranking matrices without re-execution.

452 6 Related Work

453 Our work sits at the intersection of benchmark validity, interactive-agent evaluation, verified evalu-
454 ators, partial identification, and evaluation reporting. The closest prior work usually contributes a
455 new environment, a stronger evaluator, or a benchmark-audit checklist. We address a complementary
456 question: when some completed records are not decidable under the benchmark’s own claim, how
457 should the benchmark report the uncertainty that remains?

458 **Benchmark validity and reporting.** Recent benchmark-validity work, including the Agentic
459 Benchmark Checklist, argues that agent benchmarks should make construct validity, outcome validity,
460 and reporting assumptions explicit [?]. We share that goal but provide a concrete reporting object: a
461 locked evidence contract, a counted/UNRESOLVE decomposition, and a measurement envelope.

462 **Verified and state-based interactive-agent benchmarks.** A recent wave of work has moved
463 interactive-agent evaluation toward stronger state-based or verifier-based scoring. Examples include
464 WebArena and the verified WEBARENA-VERIFIED release [? ?], APPWORLD with database-state
465 unit tests [?], the SWE-bench Verified split [?], BrowserGym [?], and ToolBench-style verifier
466 suites [?]. These projects strengthen the evaluator inside a particular domain. Our work adds a
467 cross-benchmark reporting layer and uses the strongest of these benchmarks as calibrated negative
468 controls.

469 **Other interactive-agent benchmarks used or referenced.** Other relevant environments include
470 τ^3 -bench retail [?], AGENTDOJO [?], ANDROIDWORLD [?], WORKARENA [?], and OSWORLD-
471 VERIFIED [?]. Our paper does not introduce another benchmark in this family.

472 **LLM-as-judge evaluation.** Our work is also related to LLM-as-judge evaluation [? ?]. Judge-only
473 scoring can be useful when exact checkers are unavailable, but it can also assign labels where the
474 underlying claim remains unverified. For that reason, we report judge-only results as a diagnostic
475 appendix rather than a headline comparison. The contract-drafting LLM in our study is not a judge:
476 it drafts evidence requirements before scoring and never sees agent outcomes.

477 **Benchmark refresh and contamination.** Finally, our appendix maintenance study is adjacent to
478 work on benchmark refresh and contamination [? ? ?]. Our emphasis is different. We care less about
479 how many new tasks can be proposed than about how many proposed tasks become newly countable
480 measurements under a stated evidence contract.

481 7 Conclusion

482 Interactive-agent benchmarks often report scores over executed tasks, but execution is not the same
483 as evidence-supported measurement. We introduced a reporting method that classifies completed
484 records as counted success, counted fail, or UNRESOLVE under a locked, native-aligned evidence
485 contract. Records labeled UNRESOLVE are not successes, not fails, and not deleted. They induce a
486 measurement envelope $[S/N, (S + U)/N]$, whose width is exactly the UNRESOLVE share.

487 Across AGENTDOJO, APPWORLD, WEBARENA-VERIFIED, and τ^3 -bench retail, the registered
488 pattern tests whether envelope width is concentrated in evidence-sensitive domains and narrow on
489 strong-evaluator negative controls. The broader lesson is simple: a benchmark score should be read
490 together with the records that were allowed to count, the records that remained UNRESOLVE, and the
491 evidence envelope that those records imply.

References

- [] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2024. URL <https://arxiv.org/abs/2412.05467>.
- [] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL <https://arxiv.org/abs/2406.13352>.
- [] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- [] Batu Guan, Xiao Wu, Yuanyuan Yuan, and Shaohua Li. Is your benchmark (still) useful? dynamic benchmarking for code language models, 2025. URL <https://arxiv.org/abs/2503.06643>.
- [] Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. Webarena verified: Reliable evaluation for web agents. SEA @ NeurIPS 2025 poster, 2025. URL <https://openreview.net/forum?id=94tLgxmQkN>. OpenReview non-archival submission.
- [] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.naacl-main.324/>.
- [] OpenAI. Introducing SWE-bench Verified. Blog post, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed: 2026-05-01.
- [] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [] Christopher Rawles, Sarah Clinckemaiellie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL <https://arxiv.org/abs/2405.14573>.
- [] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. Judging the judges: A systematic study of position bias in llm-as-a-judge, 2024. URL <https://arxiv.org/abs/2406.07791>.
- [] Sierra Research. τ^3 -Bench: Advancing Agent Benchmarking to Knowledge and Voice. Research release, 2026. URL <https://sierra.ai/resources/research/tau-3-bench>. Accessed: 2026-04-30.
- [] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents, 2024. URL <https://arxiv.org/abs/2407.18901>.
- [] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking

- 542 multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
543
- 544 [] Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Rethinking
545 benchmark and contamination for language models with rephrased samples, 2023. URL
546 <https://arxiv.org/abs/2311.04850>.
- 547 [] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,
548 Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica.
549 Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
550
- 551 [] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
552 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
553 web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.
554
- 555 [] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash
556 Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala
557 Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon,
558 Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy
559 Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks,
560 2025. URL <https://arxiv.org/abs/2507.02825>.

A Per-Agent Envelope Breakdown

The main tables report domain-level aggregates. This appendix reports the per-agent envelope for every (agent, domain) cell, together with case-cluster bootstrap 95% case-resampling intervals for lower endpoint, upper endpoint, width, and coverage under the eligible-case-population interpretation. The per-agent envelopes underlie the robust pairwise rankings summarized in Table ??.

Table 12: Per-agent evidence envelopes by domain. Each cell reports [LB, UB], width, and counted-only score.

Domain	Agent A	Agent B	Agent C	Agent D
AGENTDOJO	FILL	FILL	FILL	FILL
APPWORLD	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	FILL	FILL	FILL	FILL
τ^3 -bench retail	FILL	FILL	FILL	FILL

B Cost of Adopting the Method

The method is only useful if benchmark builders can actually adopt it. We track the human time required to instantiate the evidence-reporting layer on each main domain.

Predeclaration is the most expensive step but is one-time per case unit. Scoring additional records on the same case unit is cheaper because the locked contract is reused. On strong-evaluator domains, much of the work is a documentation refactor: lifting the implicit verifier contract into an explicit one.

C Contract-Drafting Details

The contract drafter is recorded as part of the experimental manifest rather than inferred from the manuscript text. In the main study, the drafter model is FILL, queried with temperature FILL using prompt version FILL. The drafter is used only to propose evidence requirements before scoring. It does not see agent identities, agent traces, native scores, or outcome labels.

The prompt template requires the drafter to separate native-aligned evidence from stronger measurement evidence. The released prompt version contains the following fields:

```
You are drafting a native-aligned evidence contract for an interactive-agent
benchmark case.

Inputs:
- task text
- official policy, if any
- evaluator code or evaluator description
- database/API/browser/file/tool schema
- trace schema and available post-run artifacts

Rules:
1. State the benchmark's native success claim.
2. Cite the source of each required evidence item:
   evaluator, task text, policy, or schema.
3. Use the fixed source hierarchy:
   evaluator semantics > task text/policy > schema constraints.
4. Do not add requirements from intuition alone.
5. If a requirement goes beyond the native claim, label it as
   stronger_measurement and keep it separate from the native_aligned claim.
6. Do not infer outcomes. Do not use agent identity, traces, or scores.

Return:
claim_text: ...
native_sources: ...
required_artifacts: ...
```

Table 13: Approximate human-time cost of instantiating the evidence-reporting method on top of an existing benchmark. Times are wall-clock minutes from a trained annotator and exclude execution compute. The contract-drafting LLM reduces first-draft time but does not replace human locking.

Domain	Draft/lock contract (min/case)	Score evidence (min/record)	Tag UNRESOLVE (min/record)	Per-domain one-time setup
AGENTDOJO	FILL	FILL	FILL	evidence schema for paired arms
APPWORLD	FILL	FILL	FILL	reuse native unit-test scaffolding
WEBARENA-VERIFIED	FILL	FILL	FILL	reuse verifier; document native-aligned structural contract
τ^3 -bench retail	FILL	FILL	FILL	policy-path manifest from official policy doc

```

606 success_rule: ...
607 fail_rule: ...
608 unresolve_rule: ...
609 claim_scope: native_aligned | stronger_measurement
610 minimality_rationale: ...
611 source_support:
612     evaluator: ...
613     task_or_policy: ...
614     schema: ...

```

616 D Additional Domains and Stress Tests

617 The main paper uses four domains because the headline claim is about evidence uncertainty, not
618 benchmark breadth. This appendix adds two further negative controls and one higher-noise stress
619 test.

Table 14: Contract-drafting and locking metadata. These fields are filled from the locked manifest and released with the review package.

Field	Value
Drafter model	FILL
Temperature	FILL
Prompt version	FILL
Visible inputs	Task text; official policy; evaluator code or description; relevant schema; trace schema; available post-run artifact types
Hidden inputs	Agent identity; agent trace; native score; outcome label; alternate view verdicts
Human lock rule	Each proposed requirement must be source-checked against official evaluator semantics, task text, policy, or schema before scoring. Unsupported requirements are removed or labeled as stronger measurement claims.
Source hierarchy	Evaluator semantics first; task text and policy second; schema constraints needed to interpret evaluator-visible state third.

D.1 ANDROIDWORLD as an additional negative control

ANDROIDWORLD is reported separately because its task-specific setup and success checking make it a useful additional negative control rather than a likely main source of evidence uncertainty. We score a fixed appendix manifest and report the same envelope decomposition. Large widths would invite the same falsification reasoning as in P1.

D.2 WORKARENA as an additional negative control

WORKARENA is reported separately for a similar reason. These results do not alter the headline claim, but they test whether the method remains stable outside the four main domains.

E OSWORLD-VERIFIED as a Higher-Noise Stress Test

We intentionally do not use OSWORLD-VERIFIED as a co-equal main domain. Its practical setup combines richer operating-system tasks with higher engineering noise, including environment volatility and evaluator complexity. Accordingly, the OSWORLD-VERIFIED appendix reports three counts for the fixed appendix manifest:

1. episodes that fail before the benchmark can execute normally because of infrastructure or pre-run problems;
2. episodes that execute but whose benchmark-side evaluator fails or is unstable; and
3. episodes that execute and evaluate but still remain UNRESOLVE under the locked evidence contract.

This keeps execution failure, evaluator failure, and evidence uncertainty separate.

F Judge-Only Scoring Diagnostic

Judge-only scoring is reported as a diagnostic rather than a headline baseline. For each judge-readable completed record, a fixed blinded judge model receives the task, locked claim, trace, and available artifacts, but not the agent identity, evidence label, or UNRESOLVE reason. The point is not to claim that judge labels are ground truth. It is to identify records where semantic trace judgment produces a verdict even though the benchmark’s stated claim remains undecidable from stored artifacts.

We report judge-only disagreement rates by domain, together with the fraction of records on which judge-only scoring assigns a success/fail label to a record that remains UNRESOLVE under the evidence rule.

G Supporting Benchmark-Maintenance Study

The main paper focuses on fixed traces. This appendix asks whether the same counted-versus-UNRESOLVE distinction also changes how benchmark maintenance is reported.

G.1 Fixed update round

We run one small fixed update round on the four main domains. Each domain contributes six raw proposals, giving 24 raw proposals total. We select three proposals per domain for materialization or execution, giving 12 selected candidates. Because selected AGENTDOJO candidates are paired, the round yields additional paired-arm episodes. The round is reported as a funnel,

proposed $\rightarrow$ selected $\rightarrow$ executed $\rightarrow$ checked $\rightarrow$ {counted, UNRESOLVE},

rather than as a raw proposal total. A proposal becomes benchmark progress only when it becomes a newly countable case unit or narrows the evidence envelope of a previously UNRESOLVE case.

Table 15: Supporting micro-update round.

Domain	Proposed	Selected	Executed	Checked	Counted	UNRESOLVE	Top UNRESOLVE reason
AGENTDOJO	6	3	6	FILL	FILL	FILL	FILL
APPWORLD	6	3	3	FILL	FILL	FILL	FILL
WEBARENA-VERIFIED	6	3	3	FILL	FILL	FILL	FILL
τ^3 -bench retail	6	3	3	FILL	FILL	FILL	FILL
Total	24	12	15	FILL	FILL	FILL	FILL

G.2 Matched-budget controls

We compare the evidence-aware update process against matched-budget controls that receive the same proposal and selection budget. The controls are one-shot generation, evidence-blind generation, and static benchmark refresh. The key question is not which control writes the most candidate tasks, but which one produces the most countable updates or the largest reduction in envelope width.

H Responsible Release Plan

Interactive-agent evaluations may touch state-changing workflows, security policies, user records, and communication channels. Public artifacts should expose enough information to make denominators and envelopes auditable without releasing exploit-enabling operational details.

- **Released, fully public.** Per-case manifests with task IDs, locked claim, evidence contract, claim-scope label, UNRESOLVE taxonomy code, envelope contributions, the rescorer source code and tests, per-domain summary statistics, robust-ranking matrices, denominator audits, and audit instructions.
- **Released under access control.** Full traces for AGENTDOJO paired arms involving prompt-injection content, and trace fragments that contain placeholder credentials, API tokens, or synthetic personal data, after credential scrubbing and sandbox re-pinning.
- **Not released.** Live-account credentials, real third-party API keys, and environment artifacts that would let a reader reproduce a side effect against a non-sandboxed service.
- **UNRESOLVE-record visibility.** Even when full traces are gated, UNRESOLVE records remain visible through case identifier, locked claim, evidence contract, taxonomy code, and envelope contribution.
- **Versioning.** The manifest, rescorer, evidence-contract template, contract-drafting prompt, and UNRESOLVE taxonomy each carry a semantic version. Reported envelopes are always paired with a (*rescorer*, *manifest*, *contract*, *taxonomy*) version tuple.

682 I Formal Definitions

683 I.1 Trajectory

684 A run produces a trajectory:

```
685 tau = (x_0, a_0, x_1, a_1, ..., x_T)
```

688 where x_t is the observed environment state or observation and a_t is the agent action.

689 I.2 Evidence function

690 An evidence function extracts evidence:

```
691 E(tau, env) -> e
```

694 where e may include logs, receipts, state diffs, hashes, tool-call records, verifier inputs, or other domain-specific measurements.

696 I.3 Oracle

697 An oracle family maps evidence to verdicts:

```
698 O_k(e, policy) -> {SUCCESS, FAIL, INCONCLUSIVE, NOT_APPLICABLE}
```

701 I.4 Counting and envelope

702 A completed record (case unit, agent) is counted only when the executed run, locked claim, and available evidence jointly satisfy the evidence rule:

```
704 Countable(case, agent, run, evidence, oracle) = true
```

707 If countable, the oracle returns SUCCESS or FAIL; otherwise the record remains UNRESOLVE with a blocking reason. For any completed record set, the reported all-record performance object is:

```
709 TOTAL = SUCCESS + FAIL + UNRESOLVE
710 COVERAGE = (SUCCESS + FAIL) / TOTAL
711 COUNTED_ONLY_SCORE = SUCCESS / (SUCCESS + FAIL)
712 LOWER = SUCCESS / TOTAL
713 UPPER = (SUCCESS + UNRESOLVE) / TOTAL
714 WIDTH = UNRESOLVE / TOTAL
```

717 J Result Macro Contract

718 The final camera-ready build should be produced from the scored manifest. The manifest-generated file `outputs/latex/results_macros.tex` should set `\resultdatatrue` and define the domain-level macros used in Tables ??–??. It should also define the contract-drafting metadata macros `\ContractDrafterModel`, `\ContractDrafterTemperature`, and `\ContractPromptVersion`. The separate row files `pairwise_margin_rows.tex`, `per_agent_rows.tex`, `cost_rows.tex`, and `update_rows.tex` populate the longer appendix tables. This contract prevents the manuscript from silently replacing missing empirical results or contract metadata with synthetic values.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- 878 • Providing as much information as possible in supplemental material (appended to the
879 paper) is recommended, but including URLs to data and code is permitted.

880 6. Experimental setting/details

881 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
882 rameters, how they were chosen, type of optimizer) necessary to understand the results?

883 Answer: **[TODO]**

884 Justification: **[TODO]**

885 Guidelines:

- 886 • The answer **[N/A]** means that the paper does not include experiments.
- 887 • The experimental setting should be presented in the core of the paper to a level of detail
888 that is necessary to appreciate the results and make sense of them.
- 889 • The full details can be provided either with the code, in appendix, or as supplemental
890 material.

891 7. Experiment statistical significance

892 Question: Does the paper report error bars suitably and correctly defined or other appropriate
893 information about the statistical significance of the experiments?

894 Answer: **[TODO]**

895 Justification: **[TODO]**

896 Guidelines:

- 897 • The answer **[N/A]** means that the paper does not include experiments.
- 898 • The authors should answer **[Yes]** if the results are accompanied by error bars, confidence
899 intervals, or statistical significance tests, at least for the experiments that support the
900 main claims of the paper.
- 901 • The factors of variability that the error bars are capturing should be clearly stated (for
902 example, train/test split, initialization, random drawing of some parameter, or overall
903 run with given experimental conditions).
- 904 • The method for calculating the error bars should be explained (closed form formula,
905 call to a library function, bootstrap, etc.)
- 906 • The assumptions made should be given (e.g., Normally distributed errors).
- 907 • It should be clear whether the error bar is the standard deviation or the standard error
908 of the mean.
- 909 • It is OK to report 1-sigma error bars, but one should state it. The authors should
910 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis
911 of Normality of errors is not verified.
- 912 • For asymmetric distributions, the authors should be careful not to show in tables or
913 figures symmetric error bars that would yield results that are out of range (e.g., negative
914 error rates).
- 915 • If error bars are reported in tables or plots, the authors should explain in the text how
916 they were calculated and reference the corresponding figures or tables in the text.

917 8. Experiments compute resources

918 Question: For each experiment, does the paper provide sufficient information on the com-
919 puter resources (type of compute workers, memory, time of execution) needed to reproduce
920 the experiments?

921 Answer: **[TODO]**

922 Justification: **[TODO]**

923 Guidelines:

- 924 • The answer **[N/A]** means that the paper does not include experiments.
- 925 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
926 or cloud provider, including relevant memory and storage.
- 927 • The paper should provide the amount of compute required for each of the individual
928 experimental runs as well as estimate the total compute.

- 929 • The paper should disclose whether the full research project required more compute
930 than the experiments reported in the paper (e.g., preliminary or failed experiments that
931 didn't make it into the paper).

932 **9. Code of ethics**

933 Question: Does the research conducted in the paper conform, in every respect, with the
934 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

935 Answer: **[TODO]**

936 Justification: **[TODO]**

937 Guidelines:

- 938 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
939 Ethics.
940 • If the authors answer **[No]**, they should explain the special circumstances that require a
941 deviation from the Code of Ethics.
942 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
943 eration due to laws or regulations in their jurisdiction).

944 **10. Broader impacts**

945 Question: Does the paper discuss both potential positive societal impacts and negative
946 societal impacts of the work performed?

947 Answer: **[TODO]**

948 Justification: **[TODO]**

949 Guidelines:

- 950 • The answer [N/A] means that there is no societal impact of the work performed.
951 • If the authors answer [N/A] or **[No]**, they should explain why their work has no societal
952 impact or why the paper does not address societal impact.
953 • Examples of negative societal impacts include potential malicious or unintended uses
954 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
955 (e.g., deployment of technologies that could make decisions that unfairly impact specific
956 groups), privacy considerations, and security considerations.
957 • The conference expects that many papers will be foundational research and not tied
958 to particular applications, let alone deployments. However, if there is a direct path to
959 any negative applications, the authors should point it out. For example, it is legitimate
960 to point out that an improvement in the quality of generative models could be used to
961 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
962 that a generic algorithm for optimizing neural networks could enable people to train
963 models that generate Deepfakes faster.
964 • The authors should consider possible harms that could arise when the technology is
965 being used as intended and functioning correctly, harms that could arise when the
966 technology is being used as intended but gives incorrect results, and harms following
967 from (intentional or unintentional) misuse of the technology.
968 • If there are negative societal impacts, the authors could also discuss possible mitigation
969 strategies (e.g., gated release of models, providing defenses in addition to attacks,
970 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
971 feedback over time, improving the efficiency and accessibility of ML).

972 **11. Safeguards**

973 Question: Does the paper describe safeguards that have been put in place for responsible
974 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
975 image generators, or scraped datasets)?

976 Answer: **[TODO]**

977 Justification: **[TODO]**

978 Guidelines:

- 979 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

1031 Guidelines:

1032 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1033 with human subjects.

1034 • Including this information in the supplemental material is fine, but if the main contribu-

1035 tion of the paper involves human subjects, then as much detail as possible should be

1036 included in the main paper.

1037 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1038 or other labor should be paid at least the minimum wage in the country of the data

1039 collector.

1040 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1041 **subjects**

1042 Question: Does the paper describe potential risks incurred by study participants, whether

1043 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1044 approvals (or an equivalent approval/review based on the requirements of your country or

1045 institution) were obtained?

1046 Answer: **[TODO]**

1047 Justification: **[TODO]**

1048 Guidelines:

1049 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1050 with human subjects.

1051 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1052 may be required for any human subjects research. If you obtained IRB approval, you

1053 should clearly state this in the paper.

1054 • We recognize that the procedures for this may vary significantly between institutions

1055 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1056 guidelines for their institution.

1057 • For initial submissions, do not include any information that would break anonymity (if

1058 applicable), such as the institution conducting the review.

1059 **16. Declaration of LLM usage**

1060 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1061 non-standard component of the core methods in this research? Note that if the LLM is used

1062 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1063 scientific rigor, or originality of the research, declaration is not required.

1064 Answer: **[TODO]**

1065 Justification: **[TODO]**

1066 Guidelines:

1067 • The answer [N/A] means that the core method development in this research does not

1068 involve LLMs as any important, original, or non-standard components.

1069 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not

1070 be described.